STA380 Exam Review

Anna Ly

Department of Mathematical and Computational Sciences
University of Toronto Mississauga

March 30, 2026

Big Picture of STA380

- **Unit 1:** Synthetic data generation; understanding on a low-level how algorithms such as `rnorm`, `rexp`, `rbeta`, `rgeom`, etc., work. The ability to generate from a variety of other uncommon distributions.
- **Unit 2:** Use the SLLN to approximate integrals and sums. Most things we calculate, i.e., expected values and variances, are essentially integrals and sums.
- **Unit 3:** Parametric bootstrapping; simulate the data and compute the statistics! (STA260, make it computational.)
- **Unit 4:** Nonparametric bootstrapping and resampling methods.
- **Unit 5:** Histograms... (Not needed for final exam; added to formula sheet for paranoid students.)
- **Unit 6:** Optimization. More advanced methods to find parameter estimates, especially for distributions with at least 2 unknown parameters.

Final Exam Q1: EM Algorithm (Unit 6)

- **Topic:** Expectation-Maximization algorithm computation.
- It is very similar to the actual practice exams. However, the EM algorithm is sensitive to small mistakes. You will likely spend most of the exam on this question.
- Take your time and pay attention to what is missing and what is not.
- Understand the differences between the **incomplete** log likelihood and the **complete log likelihood**.
- Know the differences between the **E-Step** and the **M-Step.**

Final Exam Q1: EM Algorithm (Unit 6)

Example

Consider $X_i \overset{i.i.d}{\sim} Beta(\alpha, \beta)$ where $i = 1, 2, \ldots, n$ where $n = 10^4$. Suppose we had all of the realisations except for one of them, let's say X_2. Construct the EM algorithm to solve for (α, β) and code it in R.

$$L(\alpha, \beta \,|\, \mathbf{x}) = \left(\frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)}\right)^{n-1} \prod_{i=1, i\neq 2}^{n} x_i^{\alpha-1}(1-x_i)^{\beta-1}$$

$$\begin{aligned} l(\alpha, \beta \,|\, \mathbf{x}) &= (n-1)[\ln(\Gamma(\alpha+\beta)) - \ln(\Gamma(\alpha)) - \ln(\Gamma(\beta))] \\ &\quad + (\alpha-1)\sum_{i=1, i\neq 2}^{n} \ln(x_i) + (\beta-1)\sum_{i=1, i\neq 2}^{n} \ln(1-x_i) \end{aligned}$$

We need to use an optimization method (i.e., `optim()`) on the function $l(\alpha, \beta \,|\, \mathbf{x})$ to find a starting point $(\hat{\alpha}^{(0)}, \hat{\beta}^{(0)})$.

Final Exam Q1: EM Algorithm (Unit 6)

$$l_c(\alpha, \beta \mid \mathbf{x}) = n[\ln(\alpha+\beta) - \ln(\Gamma(\alpha)) - \ln(\Gamma(\beta))] + (\alpha-1)\sum_{i=1}^{n} \ln(x_i) + (\beta-1)\sum_{i=1}^{n} \ln(1-x_i)$$

$$\begin{aligned}
&\mathbb{E}[l_c(\alpha, \beta \mid \mathbf{x}) \mid \hat{\alpha}^{(0)}, \hat{\beta}^{(0)}, \mathbf{x}_{-1}] = n[\ln(\alpha+\beta) - \ln(\Gamma(\alpha)) - \ln(\Gamma(\beta))] \\
&+ (\alpha-1)\sum_{i=1, i\neq 2}^{n} \ln(x_i) + (\beta-1)\sum_{i=1, i\neq 2}^{n} \ln(1-x_i) \\
&+ (\alpha-1)\mathbb{E}[\ln(x_2) \mid \hat{\alpha}^{(0)}, \hat{\beta}^{(0)}, \mathbf{x}_{-1}] + (\beta-1)\mathbb{E}[\ln(1-x_2) \mid \hat{\alpha}^{(0)}, \hat{\beta}^{(0)}, \mathbf{x}_{-1}]
\end{aligned}$$

We will use MC methods to compute $\mathbb{E}[\ln(x_2) \mid \hat{\alpha}^{(0)}, \hat{\beta}^{(0)}, \mathbf{x}_{-1}]$, and

$\mathbb{E}[\ln(1-x_2) \mid \hat{\alpha}^{(0)}, \hat{\beta}^{(0)}, \mathbf{x}_{-1}]$.

Final Exam Q1: EM Algorithm (Unit 6)

Let $Y_1, \ldots, Y_{m_1} \sim Beta(\hat{\alpha}^{(k)}, \hat{\beta}^{(k)})$, $Y_1, \ldots, Y_{m_2} \sim Beta(\hat{\alpha}^{(k)}, \hat{\beta}^{(k)})$. Then,

$$\mathbb{E}[\ln(X_2) \,|\, \hat{\alpha}^{(0)}, \hat{\beta}^{(0)}, \mathbf{x}_{-1}] \approx \frac{1}{m_1} \sum_{j=1}^{m_1} \ln(Y_j)$$

$$\mathbb{E}[\ln(1 - X_2) \,|\, \hat{\alpha}^{(0)}, \hat{\beta}^{(0)}, \mathbf{x}_{-1}] \approx \frac{1}{m_2} \sum_{j=1}^{m_2} \ln(1 - Y_j)$$

by the SLLN.

Final Exam Q1: EM Algorithm (Unit 6)

Thus, we let (if $\theta = (\alpha, \beta)$)

$$
\begin{aligned}
Q(\theta, \hat{\theta}) &= n[\ln(\alpha + \beta) - \ln(\Gamma(\alpha)) - \ln(\Gamma(\beta))] \\
&+ (\alpha - 1) \sum_{i=1, i \neq 2}^{n} \ln(x_i) + (\beta - 1) \sum_{i=1, i \neq 2}^{n} \ln(1 - x_i) \\
&+ (\alpha - 1) \frac{1}{m_1} \sum_{j=1}^{m_1} \ln(Y_j) + (\beta - 1) \frac{1}{m_2} \sum_{j=1}^{m_2} \ln(1 - Y_j),
\end{aligned}
$$

and we use an optimization method (i.e., `optim()`) on the function $Q(\theta, \hat{\theta})$ to find the next estimate $(\hat{\alpha}^{(k+1)}, \hat{\beta}^{(k+1)})$.

Final Exam Q2: Coding Question (Unit 4 or 6)

- **Topic:** R coding question from nonparametric bootstrapping (Unit 4) or optimization (Unit 6).
- **Unit 4:** Understand how to compute bootstrap and jackknife point estimates, bootstrap confidence intervals, difference between empirical bootstrap and percentile bootstrap, jackknife bias estimate, jackknife standard error estimate, and KS permutation test.
- **Unit 6:** Understand how to use `optimize()` and `optim()`. The differences between the two.
- *The formula sheet you bring for the exam should be to help target this question.*

Final Exam Q2: Coding Question (Unit 4 or 6)

Bootstrap Estimate

```
B <- 200 # number of bootstrap replicates
n <- nrow(dataset)

mat <- matrix(
    sample.int(n, size = n * B, replace = TRUE),
    nrow = B, ncol = n)

R <- rowMeans(
    matrix(dataset$variable[mat], nrow = B, ncol = n)
)
```

Final Exam Q2: Coding Question (Unit 4 or 6)

Jackknife Estimate

```
jack_estimate <- numeric(n)
for(i in 1:n){
  new_samp <- dataset$variable[-i]
  # can replace "function" with mean, var, median...
  jack_estimate[i] <- function(new_samp)
}
```

Final Exam Q2: Coding Question (Unit 4 or 6)

Percentile Bootstrap of the Mean

```
alpha = 0.05
# assuming R is a vector that contains all of the bootstrapped means
quantile(R, c(alpha/2, 1-alpha/2))
```

Empirical Bootstrap of the Mean

```
deltastar <- R - mean(sleepstudy$Reaction)
alpha = 0.05
q_star <- quantile(deltastar, c(1 -alpha/2, alpha/2))
# below is the confidence interval
mean(sleepstudy$Reaction) - q_star
```

Final Exam Q2: Coding Question (Unit 4 or 6)

KS Permutation Test (Part 1)

```
n <- 30
x <- samples from distribution 1
y <- samples from distribution 2
z <- c(x, y) # pooled samples

# computing the KS statistic
Fn <- ecdf(x)
Gn <- ecdf(y)
D0 <- max(abs(Fn(z) - Gn(z)))
```

Final Exam Q2: Coding Question (Unit 4 or 6)

KS Permutation Test (Part 2)

```
# Resampling the KS-test statistic
R <- 999
D <- numeric(R)
for (i in 1:R) {
  k <- sample(1:2*n, size = n, replace = FALSE)
  xi <- z[k]
  yi <- z[-k]

  Fni <- ecdf(xi)
  Gni <- ecdf(yi)

  D[i] <- max(abs(Fni(z) - Gni(z)))
}
```

Final Exam Q2: Coding Question (Unit 4 or 6)

KS Permutation Test (Part 3)

```
p <- mean(c(D0, D) >= D0)
```

Main idea of this method:

- If we have that $D_i \geq D_0$, this means the **permuted samples produce an equally or more extreme separation than the original**, meaning the original D_0 is not surprisingly large, thus failing to reject the null hypothesis.
- If we have that $D_i < D_0$, this means that the **permuted samples produce a less extreme separation than the original**, meaning that D_0 is **surprisingly** large, thus rejecting the null hypothesis.

Compute a p-value by averaging over the number of cases where $D_i \geq D$.

Final Exam Q3: Novel Proof (Unit 4)

- **Topic:** An unseen proof related to nonparametric bootstrapping and other resampling methods.
- *Practice Exams A and B contain similar "never-seen-before" proof questions — review these carefully.*
- I will not elaborate any further. **The point of this question is to provide something novel to make sure you understand the concepts instead of memorizing existing proofs from lecture.**

Final Exam Q3: Novel Proof (Unit 4)

Example

Prove that the jackknife mean is equal to the sample mean.

$$\overline{x}_{(i)} = \frac{1}{n-1} \sum_{j=1, j \neq i}^{n} x_i = \frac{1}{n-1}(n\overline{x} - x_i)$$

$$\overline{\overline{x}_{(.)}} = \frac{1}{n} \sum_{i=1}^{n} \overline{x}_{(i)} = \frac{1}{n} \sum_{i=1}^{n} \frac{1}{n-1}(n\overline{x} - x_i) = \frac{1}{n} \frac{1}{n-1}(n^2\overline{x} - n\overline{x}) = \overline{x}$$

Final Exam Q4: Unit 1

- **Topic:** May be computational or a proof.
- *You should be well-prepared from Test 1 material.*
- You should review everything from this chapter.

The Inverse Transform Method

The inverse transform method (continuous case) can be summarized as follows:

1. Find the cdf, $F_X(x)$.
2. Define the inverse function $F_X^{-1}(u)$.
3. For each random variate required:
   a. Generate a random u from $Uniform(0, 1)$.
   b. Deliver $x = F_X^{-1}(u)$.

Warning: we should only use this method if a closed form of $F_X^{-1}(x)$ exists!

Final Exam Q4: Unit 1: Recall Inverse Transform (Continuous Case)

Example

Use the inverse transform method to generate samples from the arcsine distribution:

$$f_X(x) = \frac{1}{\pi\sqrt{x(1-x)}}, \quad 0 < x < 1$$

$$F(x) = \int_0^x \frac{1}{\pi\sqrt{t(1-t)}} dt$$

Note that $u = \sqrt{t} \;\Rightarrow\; du = \frac{1}{2\sqrt{t}} dt$ and also,

$$u = \sqrt{t} \;\Rightarrow\; u^2 = t \;\Rightarrow\; \sqrt{1-u^2} = \sqrt{1-t}$$

Thus,

$$F(x) = \int_0^{\sqrt{x}} \frac{1}{\pi\sqrt{1-u^2}} 2\, du = \frac{2}{\pi}[\arcsin(\sqrt{x}) - \arcsin(0)] = \frac{2}{\pi}\arcsin(\sqrt{x})$$

Final Exam Q4: Unit 1: Recall Inverse Transform (Continuous Case)

$$u = \frac{2}{\pi}\arcsin(\sqrt{x}) \;\Rightarrow\; \sin\left(\frac{\pi u}{2}\right) = \sqrt{x} \;\Rightarrow\; F_X^{-1}(u) = \sin^2\left(\frac{\pi u}{2}\right)$$

For each random variate required,

1. Generate $u \sim \mathit{Uniform}(0, 1)$.
2. Deliver $F_X^{-1}(u) = \sin^2\left(\frac{\pi u}{2}\right)$.

```
n <- 10^5
u <- runif(n, 0, 1)
x <- (sin(pi * u/2))^2
```

The Inverse Transform Method

If we just want to generate a random variable X with pmf:

$$\mathbb{P}(X = x_i) = p_i, \quad i \in \mathbb{N}, \quad \sum_i p_i = 1,$$

then inverse transform method (discrete case) can be summarized as follows:

1. Generate a random u from $Uniform(0, 1)$.
2. Transform u into X as follows:

$$X = x_j \text{ if } F_X(x_{j-1}) < u \leq F_X(x_j)$$

3. It follows that,

$$X = \begin{cases} x_1 & u \leq F_X(x_1), \\ x_2 & F_X(x_1) < u \leq F_X(x_2), \\ \dots & \\ x_j & F_X(x_{j-1}) < u \leq F_X(x_j), \\ \dots & \end{cases}$$

Final Exam Q4: Unit 1: Recall Inverse Transform (Discrete Case)

Example

Use the inverse transform method to simulate a random sample from the logarithmic distribution with pmf:

$$p_i := \mathbb{P}(X = i) = \frac{a\theta^i}{i}, \quad i \in \mathbb{N}.$$

Where $0 < \theta < 1$ and $a = (-\ln(1-\theta))^{-1}$.

1. First, derive the following recursive formula:

$$p_{i+1} = \left(\frac{\theta i}{i+1}\right) p_i, \quad i \in \mathbb{N}.$$

2. Write the R code to simulate the algorithm.

The Inverse Transform Method

Solution.

$$p_{i+1} = \frac{a\theta^{i+1}}{i+1} = \frac{a\theta\theta^{i}}{(i+1)}\frac{i}{i} = \frac{\theta i}{(i+1)}\left(\frac{a\theta^{i}}{i}\right) = \frac{\theta i}{(i+1)}p_i$$

```
n <- 10000
u <- runif(n)
sim_vec <- rep(0, n)
theta <- 0.4 # Choice of theta is arbitrary
p_start <- (-log(1-theta))^(-1) * theta
```

The Inverse Transform Method

```
for(i in 1:n){
  j <- 1
  F_val <- p_start
  p_j <- p_start
  while(u[i] >= F_val){
    j <- j + 1
    p_j <- ((theta * (j-1))/j) * p_j
    F_val <- F_val + p_j
  }
  sim_vec[i] <- j
}
```

Acceptance-Rejection

Suppose we want to generate the random variable X with target density f using the acceptance-rejection method.

1. Find another random variable, Y with trial/candidate/envelope density g where there exists $c \in \mathbb{R}$ such that:
$$\frac{f(t)}{g(t)} < c.$$
2. For each random variate required:
   2.1 Generate y from the distribution with density g.
   2.2 Generate u from the $Uniform(0, 1)$ distribution.
   2.3 If
$$u < \frac{f(y)}{cg(y)}$$
   accept y and deliver $x = y$. Otherwise, reject y and generate a random variate again.

Final Exam Q4: Unit 1: Recall Acceptance-Rejection

Example

Use the acceptance rejection method to generate samples from the *Weilbull*$(\alpha = 2, \beta = 5)$ distribution:

$$f(x) = \frac{\alpha}{\beta}\left(\frac{x}{\beta}\right)^{\alpha-1} e^{-(x/\beta)^{\alpha}}, x \geq 0.$$

Let the trial density be *Gamma*$(\alpha = 2, \beta = 5)$.

Note that $\alpha = 2, \beta = 5$:

$$f(x) = \frac{2}{5}\left(\frac{x}{5}\right)^{2-1} e^{-(x/5)^2} = \frac{2}{5^2} x e^{-(x/5)^2}$$

$$g(x) = \frac{1}{\Gamma(2)5^2} x^{2-1} e^{-x/5} = \frac{1}{5^2} x e^{-x/5}$$

Final Exam Q4: Unit 1: Recall Acceptance-Rejection

$$h(x) := \frac{f(x)}{g(x)} = \frac{\frac{2}{5^2} x e^{-(x/5)^2}}{\frac{1}{5^2} x e^{-x/5}} = 2\exp\left\{-\left(\frac{x}{5}\right)^2 + \frac{x}{5}\right\}$$

$$h'(x) = 2\exp\left\{-\left(\frac{x}{5}\right)^2 + \frac{x}{5}\right\}\left(-\frac{2x}{5^2} + \frac{1}{5}\right)$$

the maximum occurs when $x = 5/2$. Thus,

$$c = h(5/2) = 2\exp\left\{-\left(\frac{1}{2}\right)^2 + \frac{1}{2}\right\} = 2e^{1/4}.$$

Final Exam Q4: Unit 1: Recall Acceptance-Rejection

For each random variate required:

1. Generate y from the $Gamma(\alpha = 2, \beta = 5)$ distribution.
2. Generate u from the $Uniform(0, 1)$ distribution.
3. If
$$u < \frac{2}{2e^{1/4}} \exp\left\{-\left(\frac{x}{5}\right)^2 + \frac{x}{5}\right\} = \frac{1}{e^{1/4}} \exp\left\{-\left(\frac{x}{5}\right)^2 + \frac{x}{5}\right\}$$
accept y and deliver $x = y$. Otherwise, reject y and generate a random variate again.

Final Exam Q4: Unit 1: Recall Acceptance-Rejection

```
n <- 10^5
c <- 2 * exp(1/4)
m <- ceiling(n*c)
y <- rgamma(m, shape = 2, scale = 5)
u <- runif(m)
accepted <- (u < 2 * exp(-(y/5)^2 + y/5) / c )
x <- y[accepted]
```

Mixture Models vs Convolutions

Convolutions and mixtures look similar but the represented distributions differ!

Example

Suppose $X_1 \sim N(0,1)$, $X_2 \sim N(3,1)$, and X_1 and X_2 are independent.

- Convolution representation:

$$S := 0.4X_1 + 0.6X_2$$

- Mixture representation:

$$F_X(x) := 0.4F_{X_1}(x_1) + 0.6F_{X_2}(x_2)$$

Note: for the convolution, the coefficients in front of the random variables do not necessarily have to add to one; we could have done $S := aX_1 + bX_2$ for any $a, b \in \mathbb{R}$. However, for finite mixture models, these coefficients must add to 1.

Final Exam Q5: Unit 2 — SLLN and Integration

- **Topic:** May be computational or a proof.
- *You should be well-prepared from Test 1 and Test 2 material.*
- You should review everything from this chapter.
- You don't need a rigourous analysis background, but you should understand on a **high level**:
  - SLLN and WLLN and their differences
  - Which notions of convergence are stronger: almost sure convergence, convergence in probability, convergence in distribution.

Final Exam Q5: Unit 2 — Discrete Monte Carlo

Example

Derive a Monte Carlo estimator of S below:

$$S = \sum_{i=0}^{\infty} i^2 \frac{2^i e^{-2}}{i!}$$

Identify that $\frac{2^i e^{-2}}{i!}$ resembles $Poisson(\lambda = 2)$. Thus, the algorithm is:

1. Generate $Y_1, \ldots, Y_m \sim Poisson(\lambda = 2)$
2. Compute $\frac{1}{m} \sum_{i=1}^{m} y_i^2$.

```
n <- 10^5
y <- rpois(n, lambda = 2)
mean(y^2)
```

Motivation: Antithetic Variables

Consider the variance of random variables X_1 and X_2:

$$\mathbb{V}ar\left(\frac{X_1 + X_2}{2}\right) = \frac{1}{4}\left[\mathbb{V}ar[X_1] + \mathbb{V}ar[X_2] + \mathbb{C}ov(X_1, X_2)\right]$$

Although $\mathbb{V}ar(\cdot) \geq 0$, it's possible for covariance to be negative.

Hence, variance is reduced if $\mathbb{C}ov(X_1, X_2) < 0$.

Final Exam Q5: Unit 2 — Antithetic Variables

Example

Use the antithetic variable method to derive a Monte Carlo estimate for this integral:

$$\int_2^5 e^{-x^2} dx$$

You need to ensure that the bounds of the integral is 0 to 1 so we'll need to use substitution: $y = \frac{x-2}{5-2} = \frac{x-2}{3}$ and thus:

$$\int_2^5 e^{-x^2} dx = \int_0^1 3e^{-(3y+2)^2} dy$$

Now we need to check if $g(y) = 3e^{-(3y+2)^2}$ is monotone between 0 and 1:

$$g'(y) = 3e^{-(3y+2)^2}(-2(3y+2)(3)) = -18e^{-(3y+2)^2}(3y+2)$$

Note that $g'(y) < 0$ always since for $1 > y > 0$, $e^{-(3y+2)^2} > 0, (3y+2) > 0$. Hence, the antithetic variable approach is valid.

Final Exam Q5: Unit 2 — Antithetic Variables

The algorithm is:

1. Generate $U_1, \ldots, U_{n/2} \sim Uniform(0, 1)$
2. Compute $\frac{1}{n/2} \sum_{i=1}^{n/2} \frac{g(u_i)+g(1-u_i)}{2}$

The code is as follows:

```
n <- 10^5
u <- runif(n/2)
fn <- function(u){
  3 * exp(-(3*u + 2)^2)
}
mean( (fn(u) + fn(1-u)) / 2 )
```

Q6: Unit 2 Proof or Short Answer

- **Possibility A — Proof:** Related to Unit 2 material ...
- **Possibility B — Short Answer:** Explain the difference between concept X and concept Y.
- **Possible comparison topics:**
  - Parametric vs. nonparametric bootstrap.
  - What is the point of using the inverse transformation or acceptance-rejection method if methods such as `rnorm()` exist?
  - `optim()` vs `optimize()`.
  - What is the point of the EM algorithm? Why don't we just compute the MLE?
  - Assuming a point-null alternative hypothesis, what is the issue with simulating type-II error and power?
  - Does the bootstrap or jackknife improve estimation of a point estimate?
- *Refer to practice exams an example of each type.*

Short Answer: Parametric vs. Nonparametric Bootstrap

- **Parametric bootstrap:** Assume the data come from a known distributional family (e.g., Normal, Exponential). Estimate the parameters $\hat{\theta}$ from the data, then simulate new datasets by drawing from $F(\hat{\theta})$.
- **Nonparametric bootstrap:** Make *no* distributional assumption. Instead, treat the empirical CDF $\hat{F}_n$ as the population and resample *with replacement* directly from the observed data.
- **Both methods:** Approximate the sampling distribution of a statistic by repeated simulation.

Short Answer: What is the Point of Learning Unit 1 Material?

- **Full question:** What is the point of using the inverse transformation or acceptance-rejection method if functions such as `rnorm()` exist?
- **Key idea:** Built-in R functions like `rnorm()` only cover common distributions – you may need to sample from custom or uncommon distributions.
- There are external packages out there for some uncommon distributions, but there is a possibility of bugs and not being maintained well.
- Knowing *how* random samples are generated builds intuition for debugging, extending, and understanding the limitations of simulation.

Short Answer: `optim()` vs `optimize()`

- **Full question:** What is the difference between `optim()` and `optimize()` in R, and when would you use each?
- `optimize()`**:** For *univariate* functions only. Finds the minimum (or maximum) of $f(x)$ over a specified interval $[a, b]$. Simple to use when there is one unknown parameter.
- `optim()`**:** For *multivariate* functions. Minimizes $f(\boldsymbol{\theta})$ over a vector of parameters $\boldsymbol{\theta} \in \mathbb{R}^p$. Requires an initial guess and a choice of method (e.g., `"Nelder-Mead"`, `"BFGS"`).
- One unknown parameter $\rightarrow$ `optimize()`. Two or more $\rightarrow$ `optim()`.
- **In the context of MLE:** Both can maximize a log-likelihood.

Short Answer: EM Algorithm Purpose?

- **Full question:** What is the point of the EM algorithm? Why don't we just compute the MLE directly?
- **The problem:** In the presence of unobserved variables, directly maximizing the observed-data log-likelihood $\ell(\boldsymbol{\theta})$ is either difficult or impossible.
- **What EM does:** Instead of maximizing $\ell(\boldsymbol{\theta})$ directly, EM iteratively maximizes a lower bound by working with the *complete-data* log-likelihood $\ell_c(\boldsymbol{\theta})$, which is typically much easier to optimize.

Short Answer: Issues with Simulating Type-II Error and Power?

- **Full question:** Assuming a point-null alternative hypothesis, what is the issue with simulating type-II error and power?
- **Recall:** Type-II error (β) is the probability of failing to reject H_0 when H_1 is true. Power $= 1 - \beta$.
- **The issue:** To simulate power, we need to generate data *under the alternative* H_1. But a point-null alternative (e.g. $H_1 : \mu = \mu_1$) specifies an exact value.
- In practice, the true parameter is **NEVER** exactly equal to any specific point.
- **Consequence:** Power is highly sensitive to the chosen alternative value. Simulating at one specific μ_1 gives power only for *that* value. It does not generalize to nearby alternatives.

Short Answer: Bootstrap or Jackknife Estimation?

- **Full question:** Does the bootstrap or jackknife improve estimation of a point estimate? If not, what are they useful for?
- No – neither the bootstrap nor the jackknife improves the point estimate itself. Resampling from the observed data cannot recover information that was never in the sample.
- **What they are useful for:** Quantifying the *uncertainty* of a point estimate – specifically, estimating its sampling distribution, variance, and bias.
- **Key difference:** Bootstrap $\approx$ jackknife for smooth statistics, but the bootstrap is preferred for non-smooth statistics (e.g. the median, quantiles).
- Resampling methods are tools for *uncertainty quantification*, not point estimation — the point estimate stays the same; what improves is our understanding of its variability.